

پرسش ۱ (۲ نمره) – استفاده از متغیرهای سراسری می‌تواند معایبی داشته باشد. به دو نمونه اشاره کنید و مختصراً توضیح دهید.

پرسش ۲ (۲.۵ نمره) – **تابعی بازگشتی** بنویسید که یک عدد $n+1$ رقمی مثل $\overline{a_n \dots a_1 a_0}$ را دریافت کرده این مقدار را برگرداند:

$$a_0 - a_1 + a_2 - \dots \pm a_n$$

مثلاً با دریافت ۱۲۳۴، مقداری که برمی‌گردد $2 = 4 - 3 + 2 - 1$ است. در نوشتن این تابع:

- از حلقه، آرایه، بردار و رشته استفاده نکنید.
- از تابع اضافه استفاده نکنید.
- می‌توانید پارامتر اضافه تعریف کنید.

پرسش ۳ (۲.۵ نمره) – در کد زیر ۵ خطی کامپایل را تشخیص دهید و نحوه رفع آن را توضیح دهید.

1	#include <iostream>	27	double total() {
2	#include <string>	28	double t = 0;
3	#include <vector>	29	for (int i=0; i<grades.size(); i++)
4	using namespace std;	30	t += grades[i];
5		31	return t;
6	class Person {	32	}
7	string name;	33	void print() const {
8	string nin;	34	cout << "Total grade of " << name
9	public:	35	<< "is " << total();
10	Person(string _name, string _nin)	36	}
11	: name(_name), nin(_nin) {}	37	private:
12	bool has_the_same_name(Person& p) {	38	vector<double> grades;
13	return name = p.name;	39	};
14	}	40	
15	};	41	int main() {
16		42	Student s("Ali", "123");
17	class Student : public Person {	43	if (s.has_the_same_name(
18	public:	44	new Student("ali", "124"))
19	Student(string _name, string _nin) {	45	return;
20	name = _name;	46	
21	nin = _nin;	47	s.add_grade(10); s.add_grade(12);
22	}	48	s.print();
23	void add_grade(double g) {	49	const Student t("Mahnaz", "125");
24	grade.push_back(g);	50	t.print();
25	}	51	}
26		52	

پرسش ۴ (۴ نمره) – قطعه کد مقابل را در نظر بگیرید و فرض کنید در تابع process احتمال رخداد دو استثناء (exception) از تایپ‌های

ErrorEx و FatalEx وجود داشته باشد. این کد را طوری تغییر دهید که در صورت رخداد ErrorEx تلاش مجدد برای process کردن

صورت بگیرد (به تعداد نامحدود) تا موقعی که اجرای process موفقیت‌آمیز خاتمه یابد. اما اگر استثناء FatalEx رخ دهد، اجرای برنامه خاتمه

می‌یابد. در هر صورت (مستقل از رخداد یا عدم رخداد هر نوع استثناء) باید تابع clean_up یک و فقط یک بار اجرا شود.

```
int main() {
    prepare();
    process();
    clean_up();
}
```

پرسش ۵ (۲ نمره) – فرآیند ترجمه و اجرای برنامه زیر به کدامیک از دلایل زیر خاتمه می‌یابد؟ چرا؟

```
#include <iostream>
#define SIZE 1000

class A {
public:
    A() { data = new int[SIZE]; }
    A(A& a) {
        for (int i = 0; i < SIZE; i++)
            data[i] = a.data[i];
    }
    int get(int i) { return data[i]; }
private:
    int* data;
};

int main() {
    A x;
    A y = x;
    std::cout << x.get(10);
}
```

الف) به طور کامل ترجمه و اجرا می‌شود
ب) خطای ترجمه: پارامتر a باید ثابت (const) تعریف شود
پ) خطای ترجمه: عملگر = برای کلاس A تعریف نشده
ت) خطای ترجمه: دسترسی به فیلد خصوص a توسط شیء دیگر
ث) خطای زمان اجرا: دسترسی غیرمجاز به حافظه
ج) خطای زمان اجرا: آزاد کردن دوباره‌ی حافظه

پرسش ۶ (۲ نمره) – فرض کنید قرار است برنامه‌ای برای مدیریت رزرو بلیت‌های یک خط هواپیمایی داشته باشیم. به این منظور چهار کلاس به شرح زیر تعریف شده‌اند. در اینجا تمرکز ما روی تعریف رابطه‌های بین این کلاس‌ها است در نتیجه اعضای دیگر آنها را حذف کرده‌ایم.

```
#include <vector>
using namespace std;

class Flight {
};

class Reservation {
    Flight* flight;
    Customer* customer;
};

class Customer {
    vector<Reservation*> reservations;
};

class System {
    vector<Customer*> customers;
};
```

الف) به جز نداشتن متد main این تعاریف یک خطای کامپایل دارند. آن را مشخص کنید و نشان دهید چطور می‌توان آن را رفع کرد.

ب) کلاس System شامل مجموعه‌ای از مشتریان است که در قالب بردار customers تعریف شده. چرا اعضای این بردار را از تایپ Customer* و نه Customer تعریف کرده‌ایم؟

ج) طراح این برنامه تصمیم گرفته System مجموعه پروازها (Flight) را در خود ذخیره نکند. دو نقطه ضعف برای این تصمیم بیان کنید، یکی از وجه درستی کارکرد برنامه و یکی از جنبه سرعت اجرا.

د) با طرح فعلی اگر بخواهیم تمام مشتریانی که یک پرواز را رزرو کرده‌اند را پیدا کنیم چه مراحل باید انجام شوند؟ برای بهبود این کار چه تغییری در طرح پیشنهاد می‌کنید؟ به تبعات منفی احتمالی پیشنهاد خود اشاره نمایید.

پرسش ۷ (۵ نمره) – برای نگهداری ساعت‌های تشکیل کلاس‌های درس دانشگاه، کلاسی به نام `WeeklySchedule` تعریف کنید. نحوه استفاده از این کلاس در قطعه‌کد زیر نمایش داده شده است:

```
WeeklySchedule ws1;
ws1.addTimeSlot(SAT, 7,30, 9,00);
ws1.addTimeSlot(MON, 7,30, 9,00);
ws1.addTimeSlot(TUE, 12,30, 14,00);

WeeklySchedule ws2;
ws2.addTimeSlot(SUN, 10,30, 12,00);
ws2.addTimeSlot(TUE, 10,30, 13,00);

cout << ws1.conflicts(ws2) << endl;
```

فرض کنید ثابت‌های روزهای هفته `SAT`, `SUN`, ... تعریف شده‌اند. متد `conflicts` تشخیص می‌دهد که آیا برنامه‌های زمانی تعریف‌شده با یکدیگر تلاقی دارند یا خیر. در تعریف این کلاس قواعد زیر را کنترل کنید و در صورت نقض آنها، اجرای برنامه را خاتمه دهید:

- تشکیل جلسات کلاس‌ها روز جمعه مجاز نیست
 - حداقل طول مدت یک جلسه کلاس ۳۰ دقیقه است
 - اعداد ساعت و دقیقه باید معتبر باشند
 - ساعت شروع پیش از ساعت پایان باشد
 - هیچ دو جلسه‌ای از یک کلاس با یکدیگر تلاقی ندارند
- دقت نمایید که همپوشانی یک دقیقه‌ای ابتدا و انتهای یک جلسه تلاقی محسوب نمی‌شود. به عنوان مثال بازه‌های ۷:۳۰ تا ۹:۰۰ و ۹:۰۰ تا ۱۰:۳۰ با یکدیگر تلاقی ندارند.

در نوشتن این کلاس به نکات زیر توجه نمایید:

- حتماً برای نگهداری اجزاء داخلی `WeeklySchedule` از کلاس‌های کمکی استفاده کنید. (حداقل دو کلاس کمکی اضافه)
- متدهای کلاس‌ها را خوانا و تا حد امکان کوتاه تعریف کنید. به عنوان یک معیار، در راه‌حل طراح سؤال، بدنه‌ی هیچ متدی بیش از ۴ خط نیست.
- تمام فیلدهای کلاس‌ها را خصوصی تعریف کند و برای هیچ‌یک از آنها `getter` تعریف نکنید.